

FinSight AI

SEC Filing Intelligence Platform

Project Report — Architecture, Stages & Rationale

2,841,255

SEC Filing Chunks

9

AI Tools

6

Data Sources

473

S&P 500 Tickers

Bloomberg-grade financial intelligence | Open source | Self-hosted
AWS Bedrock · PostgreSQL 16 · pgvector · Next.js

Project Overview

FinSight AI is a full-stack financial intelligence platform built to answer questions that Bloomberg charges \$25,000/seat/year for — using entirely open-source tools and publicly available SEC data. The system ingests the complete corpus of S&P 500 SEC filings, structures them into a hybrid retrieval engine, and connects them to a Claude-powered agentic reasoning layer that synthesizes filing evidence with stock prices, XBRL financials, earnings call transcripts, and news sentiment.

The platform is built in 7 phases, each solving a specific problem in the data → intelligence pipeline. This document explains every stage: what was built, the technical decisions made, and why those decisions were the right ones.

The Problem

Financial research has two failure modes: keyword search (Bloomberg) finds documents containing exact terms but misses semantic relationships; and LLMs without retrieval hallucinate numbers and lack access to actual filing data. FinSight solves both: hybrid retrieval surfaces the right documents, and Claude synthesizes them into grounded, cited answers.

Architecture at a Glance

UI Layer	Next.js + TypeScript + Tailwind — chat interface with streaming, charts, source panel
API Layer	FastAPI — SSE streaming, Redis cache, multi-model routing
Agent Layer	Claude (AWS Bedrock) — autonomous 9-tool agentic loop
Retrieval Layer	Hybrid BM25 + dense search → Reciprocal Rank Fusion
Data Layer	PostgreSQL 16 + pgvector — 6 tables, 2.84M chunks + structured data

Data Sources

SEC EDGAR filings	10-K, 10-Q, 8-K — 2,841,255 chunks from 38,183 + 138,030 filings
EDGAR XBRL	Company Facts API — 14 financial metrics per ticker/period
EDGAR Submissions	Company metadata — name, SIC, exchange, fiscal year end
Stock prices	473 S&P 500 tickers — 4,095,806 OHLCV rows, 1962–2026 (yfinance/S3)
News + sentiment	RSS feeds — 473 tickers, FinBERT GPU sentiment scoring
Earnings calls	Transcripts extracted from 8-K filings (Q&A + management remarks)

Stage 1 — SEC Filing Acquisition

downloader.ipynb · download_8k.py

The first challenge is simply getting the data. The SEC's EDGAR system contains millions of filings, but they are spread across a hierarchical index structure. We needed all 10-K, 10-Q, and 8-K filings for every S&P 500 company — going back as far as EDGAR records allow.

What Was Built

- `downloader.ipynb` — crawls SEC EDGAR full-text index for all S&P 500 tickers, downloads HTML filings, uploads to S3.
- `download_8k.py` — downloads 8-K filings from S3 `raw_8k/` bucket (170,531 objects → 138,030 unique HTML files).

Technical Decisions

Rate limiting	Token-bucket <code>RateLimiter</code> enforces 10 req/sec (SEC mandated). Violations result in IP bans.
S3 for raw storage	HTML files average 2–5 MB. Local storage would require ~500 GB. S3 is cheap, durable, and parallelisable.
8-K file selection	Each 8-K folder contains multiple files. Priority: <code>primary-document.html</code> → <code>.txt</code> → any <code>.html/.htm</code> . This handles SEC's inconsistent naming.
Idempotency	S3 <code>head_object</code> check before each upload — re-running the downloader skips already-uploaded files.
Parallelism	8 threads (10-K/10-Q) · 16 threads (8-K, I/O bound). More threads would violate rate limits.

Why These Filing Types?

- 10-K (annual): Management Discussion & Analysis, Risk Factors, full year financials — the richest qualitative disclosure.
- 10-Q (quarterly): Interim financials and risk updates — 3x more frequent than 10-K.
- 8-K (current): Filed within 4 days of any material event — earnings releases, M&A, CEO changes, lawsuits. Highest-signal for time-sensitive queries.

Results

10-K + 10-Q files	38,183 HTML files on S3
8-K files	138,030 HTML files local (from 170,531 S3 objects)

Stage 2 — Document Preprocessing

`convert_to_md.py`

Raw SEC HTML is not directly usable for NLP. It contains HTML tags, JavaScript, CSS stylesheets, and critically — iXBRL/XBRL financial taxonomy data embedded directly in the document. If left in, the chunker would embed thousands of machine-readable taxonomy identifiers rather than human-readable financial text.

What Was Built

- `convert_to_md.py` — converts HTML → clean Markdown using `selectolax` for fast HTML parsing.
- iXBRL stripper — detects and removes `ix:header` and `ix:hidden` blocks using `str.find()` boundary detection.
- Junk detector — three-stage filter: XBRL namespace patterns, average word length > 20 chars, English character ratio < 15%.
- Section header detector — promotes Item 1., PART I, RISK FACTORS etc. to `##` markdown headers.

Technical Decisions

<code>selectolax</code> over BeautifulSoup	5–10x faster HTML parsing. For 38k+ files, this matters.
<code>str.find()</code> for iXBRL	Regex catastrophically backtracks on multi-MB files with nested XBRL. Boundary detection is $O(n)$.
<code>FORCE_RERUN=False</code>	When adding 8-K on top of existing 10-K/10-Q corpus, skip already-converted files. Saves hours.
16 CPU workers	Preprocessing is CPU-bound (HTML parsing, regex). True parallelism via multiprocessing.
Section headers preserved	Downstream chunking uses section names to label each chunk — enables metadata-filtered retrieval.

Stage 3 — Text Chunking

chunk.py

SEC 10-K filings can be 200–400 pages. Language models have context windows. We need to split documents into chunks small enough to embed meaningfully and retrieve precisely, while preserving enough context for Claude to synthesize answers.

What Was Built

- chunk.py — section-aware sliding window chunker. Splits on markdown headers first, then applies word-window chunking within each section.
- INCREMENTAL mode — queries `SELECT DISTINCT source_file` at startup and skips any file already in the DB.
- Batch insert with `execute_values` — 500 rows per PostgreSQL `INSERT`, committed immediately after each file batch.

Key Parameters and Why

Window: 1,024 words	Long enough to capture a complete financial argument; short enough for meaningful embeddings.
Overlap: 200 words	Prevents ideas split across chunk boundaries from being lost. ~20% overlap is standard.
Min length: 50 chars	Filters out section headers and whitespace that produce useless embeddings.
24 CPU workers	Chunking is CPU-bound. <code>maxtasksperchild=20</code> bounds per-worker memory growth.
Batch size: 2,000 files	Insert after each batch then <code>gc.collect()</code> — peak RAM proportional to one batch, not full corpus.

Why Section-Aware Chunking Matters

A chunk labelled 'Risk Factors' with its section name stored in the DB produces a more discriminative embedding than the same text with no label. The model can attend to the section context. This enables queries like 'search only in Risk Factors sections' — a metadata filter that dramatically improves precision.

Stage 4 — Semantic Embedding

embed.py

Each text chunk must be converted into a 384-dimensional vector that captures its semantic meaning. These vectors enable similarity search — finding chunks whose meaning is closest to a query, even if they share no exact words.

What Was Built

- `embed.py` — streams un-embedded rows from PostgreSQL (WHERE embedding IS NULL LIMIT 256), encodes on GPU, updates rows in place.
- IVFFlat index — built after all 2.84M embeddings are populated (not during, which would be 100x slower).

Why all-MiniLM-L6-v2?

384 dimensions	Small enough for fast ANN search; large enough for high-quality representations.
Distilled from MiniLM	Trained on 1 billion sentence pairs. Strong retrieval quality relative to size.
Inference speed	~4,000 chunks/second on GPU. Critical for 2.84M chunks.
vs BERT-large/E5	80-90% of retrieval quality at 4x the throughput. Right tradeoff for this corpus size.

Why IVFFlat?

IVFFlat partitions the vector space into 100 Voronoi cells. At query time, only the nearest cells are searched — reducing retrieval from $O(n)$ to $O(n/\text{lists})$. For 2.84M vectors with `lists=100`, this gives ~10ms query latency vs ~500ms for exact search. The 1% accuracy tradeoff is acceptable for RAG.

Why Separate from `chunk.py`?

- `chunk.py` runs on CPU (no GPU needed). `embed.py` runs on CUDA. Different resource requirements.
- If embedding is interrupted, WHERE embedding IS NULL picks up exactly where it left off.
- `chunk.py` can finish hours before `embed.py` starts — pipeline stages are fully decoupled.

Results

Total chunks embedded	2,841,255 (10-K + 10-Q + 8-K)
Embedding dimensions	384
Index type	IVFFlat, cosine distance, lists=100

Stage 5 — Structured Financial Data

`ingest_xbri.py` · `ingest_company_info.py` · `ingest_stocks.py`

Semantic search over filing text is powerful for qualitative questions. But 'What was Apple's exact revenue in FY2024?' should not require reading chunks of text — it should return a precise number from a structured table. This stage builds the structured data layer that complements the vector store.

XBRL Financials (`ingest_xbri.py`)

EDGAR's Company Facts API returns all XBRL-tagged financial values for any company as structured JSON. We extract 14 key metrics per filing period.

Income Statement	revenue, gross_profit, operating_income, net_income, eps_basic, eps_diluted, rd_expense
Balance Sheet	total_assets, total_liabilities, stockholders_equity, cash, long_term_debt
Cash Flow	operating_cash_flow, capex, free_cash_flow (computed)
Computed margins	gross_margin, operating_margin, net_margin
Multi-tag fallback	Revenue alone has 5+ XBRL tags across different companies/time periods. First tag with data wins.
Duration filtering	Quarterly facts must span 60-120 days to exclude YTD cumulative figures.

Company Info (`ingest_company_info.py`)

The EDGAR Submissions API returns company metadata. This powers the `get_company_info` tool — when asked 'what sector is Nvidia in?' Claude calls this instead of searching filing text.

- Fields: name, CIK, SIC code, SIC description, state of incorporation, fiscal year end, exchange, filer category.
- Upsert on conflict — re-running updates stale data.

Stock Prices (`ingest_stocks.py` + `refresh_stocks.py`)

- Initial load: 473 ticker CSVs from S3 (yfinance format, skiprows=2 for multi-header).
- 4,095,806 rows — daily OHLCV, 1962–2026.
- `refresh_stocks.py`: incremental update — finds last stored date per ticker, downloads only new rows via yfinance.

Stage 6 — Hybrid Retrieval (BM25 + Dense + RRF)

Why keyword search alone fails — and why dense search alone isn't enough

The Problem with Pure Dense Search

Dense semantic search (pgvector cosine similarity) excels at conceptual queries — 'companies with supply chain concentration risk' — but fails on exact identifiers. If a user asks about 'ASC 842' (a lease accounting standard) or 'Item 1A', the embedding model may not weight these tokens heavily enough to surface the right chunks.

The Problem with Pure BM25

BM25 (keyword frequency-based retrieval) excels on exact matches but fails on semantic queries. 'Companies that discussed being vulnerable to interest rate changes' won't match filings that use the phrase 'exposed to monetary policy fluctuations'.

The Solution: Reciprocal Rank Fusion

RRF combines ranked result lists from both retrieval methods. For each document, its RRF score = $1/(k + \text{rank_dense}) + 1/(k + \text{rank_bm25})$, where $k=60$ is a smoothing constant. Documents that appear in both lists rank highest; documents exclusive to one list still appear based on their individual rank. This gives the best of both worlds.

Implementation

Dense path	all-MiniLM-L6-v2 → pgvector IVFFlat cosine search → top-100 candidates
BM25 path	PostgreSQL tsvector GENERATED ALWAYS AS STORED + GIN index + websearch_to_tsquery
Fusion	FULL OUTER JOIN on chunk ID → RRF score → ORDER BY rrf_score DESC → top-k
Migration	add_hybrid_search.py — adds fts column + GIN index in one transaction
k parameter	60 — standard RRF constant, balances precision vs recall

Why This Matters in Practice

- Query 'AAPL revenue guidance 2024': BM25 finds exact keyword matches in 8-K filings; dense finds semantically similar content about forward guidance.
- Query 'supply chain concentration risk': Dense dominates — BM25 only helps if the exact phrase appears.
- Query 'ASC 606 revenue recognition impact': BM25 is critical — 'ASC 606' is a specific code that must match exactly.
- Combined: ~15-25% better recall on financial queries vs either method alone.

Stage 7 — Agentic Architecture (9-Tool Claude Agent)

Why tool-use over fixed RAG pipelines — and how multi-model works

Why Agentic RAG vs Fixed Pipeline?

A fixed RAG pipeline always retrieves from the same source in the same way. A question like 'Compare Apple and Microsoft's FCF margins and tell me what management said about capital allocation' requires: XBRL data for both tickers, a chart, and earnings call transcripts — three different tools in sequence. A fixed pipeline cannot do this. Claude with tool-use can.

Claude autonomously decides which tools to call, in what order, and whether to call multiple tools before answering. This makes the system genuinely intelligent rather than a thin wrapper around a vector search.

The 9 Tools

search_filings	Hybrid BM25+dense search over 2.84M filing chunks — qualitative context
get_financials	XBRL structured numbers: revenue, EPS, margins, FCF, debt — exact figures
get_company_info	Company metadata: name, SIC industry, exchange, fiscal year end
get_earnings_call	Full earnings transcript from 8-K (management remarks + Q&A)
get_stock_data	OHLCV price history, 52-week stats, 473 tickers, 1962-2026
get_news_sentiment	FinBERT-scored news headlines, sentiment summary per ticker
get_filing_price_impact	Stock price $\pm N$ days around a filing date — temporal correlation
get_risk_drift	Cosine similarity between risk sections across consecutive 10-Ks
render_chart	Pass-through: structured data → interactive chart in UI

Multi-Model Support

The agent supports 5 models via AWS Bedrock. Anthropic models use the Anthropic SDK (native tool_use). Non-Anthropic models (Amazon Nova, Google Gemma) use the Bedrock Converse API with graceful fallback if the model doesn't support tool calling.

Claude Sonnet 4.6	Default — balanced speed and intelligence, full tool support
Claude Opus 4.6	Most capable — best for complex multi-step reasoning
Claude Haiku 4.5	Fastest — good for simple queries
Amazon Nova 2 Lite	Amazon's lightweight model — Converse API
Google Gemma 3 27B	Open-weight model — Converse API

Why AWS Bedrock vs Direct Anthropic API?

- AWS Bedrock provides a single endpoint for multiple model providers — no vendor lock-in.
- Bedrock handles request signing, rate limiting, and compliance at enterprise scale.
- The Converse API provides a unified interface across all Bedrock models.
- AnthropicBedrock client is a drop-in replacement for the standard Anthropic client.

Stage 8 — API Layer (FastAPI) + UI (Next.js)

Streaming, caching, and the frontend architecture

FastAPI — Why SSE over WebSockets?

Server-Sent Events (SSE) were chosen over WebSockets for streaming because: SSE is unidirectional (server→client) which matches the use case perfectly; SSE works through HTTP/1.1 without special infrastructure; and SSE reconnects automatically on disconnect. The agent streams token-by-token as Claude generates them, so the user sees the answer building in real time.

Redis Caching Strategy

Identical queries are cached in Redis with TTLs tuned to data freshness:

Filing queries	24 hours — SEC filings don't change once filed
Stock queries	1 hour — prices update daily, not minute-by-minute
News queries	30 minutes — news is most time-sensitive
Cache key	SHA-256 of normalized (lowercase, stripped) query string
Fallback	Redis unavailable → serve uncached (graceful degradation)

Next.js UI Components

Chat.tsx	SSE streaming chat — reads token stream, renders markdown in real time
AppShell.tsx	Main layout — chat + sidebar + source panel in resizable columns
Sidebar.tsx	Conversation history, model selector, new chat
SourcePanel.tsx	Displays tool calls made + source chunks with ticker/date/section
StockChart.tsx	Candlestick/line chart rendered when <code>get_stock_data</code> is called
SentimentChart.tsx	Sentiment distribution bar chart for news sentiment
DynamicChart.tsx	Renders <code>render_chart</code> tool output — bar, line, grouped bar, pie
lib/api.ts	SSE client — parses event stream, dispatches text/sources/chart events

Why Next.js?

- App Router with React Server Components — fast initial page load.
- TypeScript throughout — type safety for the SSE event protocol.
- Tailwind CSS — rapid styling without a component library dependency.
- Geist font — clean, modern monospace-friendly typography.

Stage 9 — News, Sentiment & Earnings Calls

news_ingest.py · sentiment.py · ingest_earnings.py

Filings are historical — they describe what already happened. To answer 'What is the current market sentiment around Nvidia?' or 'What did management say about AI infrastructure last quarter?', we need two additional data sources: real-time news and earnings call transcripts.

News Ingestion (news_ingest.py)

- RSS feeds for all 473 tickers — Google Finance, Yahoo Finance, MarketWatch.
- 50 articles/ticker/run maximum — prevents runaway ingestion.
- feedparser + requests — handles Atom and RSS 2.0 formats.
- ON CONFLICT DO NOTHING — idempotent; re-running never creates duplicates.

Sentiment Scoring (sentiment.py)

FinBERT (ProsusAI) is a BERT model fine-tuned on financial text from Reuters and financial analyst reports. It outperforms general-purpose sentiment models on financial language — where 'the company missed expectations' is strongly negative but 'the company maintained guidance' is positive.

Model	ProsusAI/finbert — fine-tuned on financial news
Output	positive / negative / neutral label + confidence score
Batch size	256 articles per GPU inference batch
Resumable	WHERE sentiment_label IS NULL — skips already-scored rows

Earnings Calls (ingest_earnings.py)

Rather than building a separate scraper, earnings call transcripts are extracted from the 8-K filings already in the database. Many companies file their earnings call transcripts as 8-K Item 7.01 or 8.01 events.

- Keyword matching on chunk_text: 'earnings call', 'conference call', 'earnings per share', 'Q&A session', etc.
- Groups matching chunks by source_file (accession number), concatenates in chunk_index order.
- Stores full transcript text (up to 80K chars) in earnings_calls table.
- Fiscal quarter inferred from filing month — approximate but sufficient for retrieval.

Daily Refresh (daily_refresh.py)

For a production deployment, daily_refresh.py chains all three refresh jobs: stock prices → news ingestion → sentiment scoring. Designed to run as a cron job at 5pm weekdays (after US market close). For a university demo, run manually before presenting.

Results & Final State

Data Scale

filing_chunks	2,841,255 rows — 10-K, 10-Q, 8-K from 473 S&P 500 companies
stock_prices	4,095,806 rows — 473 tickers, daily OHLCV, 1962–2026
financials	XBRL annual + quarterly — 14 metrics per period per ticker
company_info	473 companies — SIC, exchange, fiscal year end
news_articles	RSS headlines + FinBERT sentiment — up to 50/ticker/run
earnings_calls	Transcripts extracted from 8-K filings — up to 80K chars each

What the System Can Answer

The following query types are enabled by the 9-tool architecture — none are possible with keyword search alone:

- **BENCHMARKING:** 'What are the gross margins and R&D spend ratios for Salesforce, HubSpot, and Workday over 3 years?' (get_financials x3 + render_chart)
- **RISK INTELLIGENCE:** 'What new risks did Meta disclose in 2024 that didn't appear in 2022?' (get_risk_drift)
- **MARKET REACTION:** 'How did Tesla's stock react in the 10 days after their last 10-K?' (get_filing_price_impact + search_filings)
- **MANAGEMENT TONE:** 'What did Nvidia's CEO say about AI infrastructure demand across 3 earnings calls?' (get_earnings_call x3)
- **SECTOR TRENDS:** 'Which sectors saw the biggest increase in AI risk disclosures in 2024?' (search_filings, no ticker filter)
- **TEMPORAL ANALYSIS:** 'Show Apple's FCF margin trend since 2018 alongside stock performance' (get_financials + get_stock_data + render_chart)

Key Engineering Decisions — Summary

chunk.py separated from embed.py	CPU vs GPU separation — flat memory, independent resumability
embed.py: WHERE embedding IS NULL	Natural checkpoint — restart from any interruption point
INCREMENTAL=True in chunk.py	8-K top-ups never touch existing 10-K/10-Q chunks
IVFlat built after bulk insert	Building during insert is 100x slower; build once at end
RRF k=60	Industry standard — smooths rank distribution, avoids over-weighting top-1
Generated tsvector column	Auto-updates on INSERT/UPDATE — zero maintenance BM25 index
ON CONFLICT DO NOTHING everywhere	All ingestion is idempotent — re-runnable without corruption
Agentic loop vs fixed RAG	Claude decides tool sequence — handles multi-source queries naturally
SSE over WebSockets	Simpler infrastructure, auto-reconnect, perfect for unidirectional streaming
Redis TTL by data type	Matches cache lifetime to data freshness — correctness + performance

Technology Stack

Language	Python 3.13 (backend) · TypeScript (frontend)
Database	PostgreSQL 16 + pgvector extension

Embedding model	all-MiniLM-L6-v2 (sentence-transformers) — 384 dimensions, CUDA
Vector index	IVFFlat cosine, lists=100
BM25 index	PostgreSQL GIN on tsvector — websearch_to_tsquery
LLM / Agent	Anthropic Claude (Sonnet 4.6 default) via AWS Bedrock
API framework	FastAPI + Uvicorn — SSE streaming
Cache	Redis — query-level caching with smart TTLs
UI framework	Next.js 15 + React + Tailwind CSS + Geist
Cloud storage	AWS S3 — raw HTML filings (~180 GB)
Sentiment model	FinBERT (ProsusAI) — positive/negative/neutral per headline
Stock data	yfinance (initial + incremental refresh)
XBRL data	EDGAR Company Facts API (data.sec.gov)